

Versioning Insights

Versioning Insights — Planning Environments and Simulations

Date: 2026-04-18 **Status:** Design decision captured, not yet implemented **Scope:** How the Flux Scheduler supports committed production plans, in-progress planning work, and ephemeral feasibility simulations

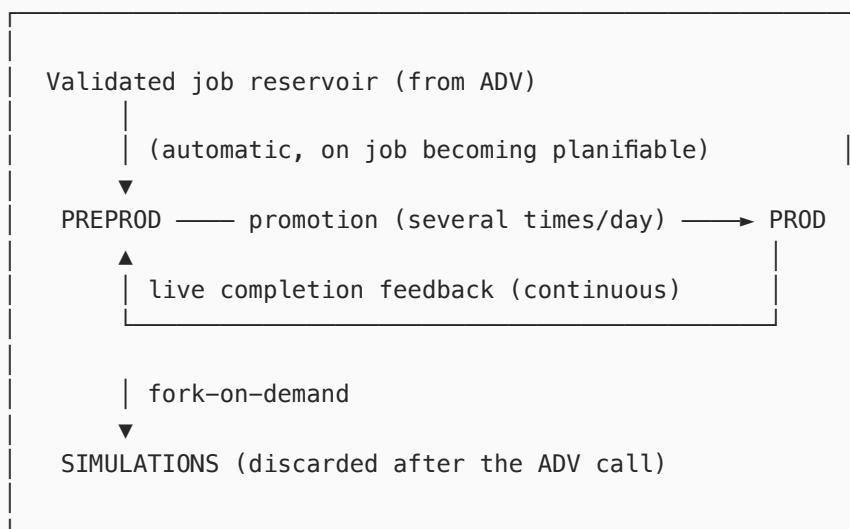
1. The Decided Solution

1.1 Three planning environments

The system separates planning work into three environments with distinct semantics. They are not variants of a generic “scenario” — they are different roles with different rules.

Environment	Role	Mutability	Cardinality	Lifetime
Prod	Frozen snapshot of what the shop floor is executing	Read-only, except live completion feedback	1 per company	Replaced on each promotion
Preprod	Planning workspace — the current app IS this	Fully mutable, free planning	1 per company	Persistent
Simulation	Isolated copy of preprod used by sales assistants (ADV) for feasibility quoting	Fully mutable in its bubble	N (one per client call)	Short-lived (15 min typical, TTL-bounded)
Archive	Auto-snapshot of preprod state created immediately before each promotion to prod	Read-only	N, grows indefinitely	Kept forever (for audit and undo)

1.2 The three data flows



Three channels with explicit direction and cadence:

1. **Reservoir → Preprod** — a job entering the planifiable state (validated by ADV, BAT received, BAT deadline fixed) is automatically added to the preprod with default planning values. No user action required.
2. **Preprod ↔ Prod** — promotion flows downward (manual, by the lead planner, several times per day); completion feedback flows upward (continuous, automatic, only touches advancement fields).
3. **Preprod → Simulations** — an ADV forks a bubble from the current preprod state when taking a client call; the bubble dies on session close or after a TTL. Never promoted.

1.3 Behaviour of a simulation

A simulation is a feasibility tool, **not a staging area**. When the ADV answers “yes, we can fit it” to a client:

- The simulation itself is discarded.
- The real job is created through the normal JCF flow and enters the reservoir through the standard path.
- The planning decisions taken inside the simulation are **not** imported into the preprod.

This decoupling keeps a single source of truth for planning (the preprod) and avoids the class of bugs where a simulation’s hypothetical state accidentally escapes into reality.

1.4 Promotion mechanics

When the lead planner promotes the preprod to prod:

1. An archive is automatically created from the current prod (named e.g. `Prod 18/04/2026 16:24 (auto-archive before promotion)`).
2. The preprod’s planning state replaces the prod’s planning state entirely.
3. **Exception:** live completion status on tiles is preserved from the current prod — it reflects the reality of shop floor execution and must not be overwritten by a potentially stale preprod value.
4. A 5-minute undo window is available; after that, promotion is final (but always restorable from archive).

Emergency modifications (machine breakdown, etc.) follow the same path: edit in preprod, re-promote. No hot-patch path is provided, because promotion is cheap and frequent in this setup.

1.5 Architectural principles

- **Full copies, not overlays.** Each environment carries a complete copy of all planning entities. An overlay/delta approach was considered and rejected because the use cases do not benefit from it (single-user per environment, short-lived simulations, frequent promotions).

- **scenarioId on every planning entity.** Jobs, Tasks, TaskAssignments, Operators, Machines, Groups, Productivities, Schedule state — all carry a `scenarioId` foreign key. The Prod is one value of this id; the Preprod is another; each simulation is its own.
- **The Rust scheduling engine is scenario-aware.** Every API request carries a `scenarioId`. The engine reads from and writes to that scenario's data only. No accidental cross-scenario reads or writes.
- **URL-scoped context in the web app.** `scenarioId` lives in the URL (path or query parameter), not in `localStorage`. This is what allows two browser tabs to hold different contexts independently — a prerequisite for the ADV simulation flow where multiple tabs may coexist.

1.6 Non-goals (explicitly excluded from the design)

To prevent scope creep, these were considered and rejected:

- **Multiple named scenarios with colors and branching.** The real use cases need two fixed environments plus ephemeral simulations, not a rich branching system.
- **Approver role distinct from planner.** The lead planner (`chef d'atelier`) is the approver by definition of their role. A separate “approver” entity would add ceremony without value.
- **Cross-editing of scenarios by multiple planners.** There is one planner per company on the preprod; multi-user collaboration on the same environment is out of scope.
- **Staleness / merge-conflict machinery.** With a single preprod being continuously synced from prod, the staleness problem classical to branching systems does not arise.
- **A generic “scenario management” UI with lists, filters, tags.** The user never manages a list of scenarios; the environments are fixed, the simulations are ephemeral.
- **Overlay/delta data model.** Rejected after extensive exploration — see section 2.5.

1.7 UX principles

- **One workspace per role.** The lead planner lives in the preprod. The shop floor operators live in the prod (write-limited to completion feedback). The ADV lives in their simulation tab during a client call. No ambiguity on “which world am I in” because each role has one answer.
- **Preprod/Prod toggle, not a switcher.** The lead planner's UI has a two-state toggle between preprod (mutable, their habitat) and prod (read-only view of what's running). No dropdown, no multi-scenario switcher.
- **New simulation opens in a new tab.** The ADV clicks “Simulate a new job” (e.g. from within the JCF flow); a new browser tab opens with an isolated bubble. The tab's chrome is visually distinct (a tint or a fine colored band) to signal non-production context. Closing the tab effectively ends the simulation.
- **Promotion is a distinct ritual.** A dedicated flow with a diff preview, an explicit confirmation (the checkbox or name re-entry), the auto-archive creation, and a post-promotion undo

toast. The gravity of the action is reflected in the friction of the UI.

1.8 Implementation phases

A sketch of how to sequence the work, without committing to timelines:

1. **Data model migration.** Add `scenarioId` to every planning entity. Create the `scenario` table with type (`prod` | `preprod` | `simulation` | `archive`) and status fields. Bootstrap the current planning as the initial preprod.
 2. **Scenario-aware API.** Make every PHP API endpoint accept and honor a `scenarioId` . Make the Rust scheduling engine accept a `scenarioId` in input and scope all its reads/writes accordingly.
 3. **Sync workers (backend).** Two background processes: (a) pushes newly-planifiable jobs from the reservoir into the preprod; (b) syncs completion status from prod to preprod continuously.
 4. **Promotion flow.** The action that clones the preprod state into prod, preserves prod's live completion, and creates an archive. Diff preview, confirmation, undo window.
 5. **Prod read-only view.** A UI that renders the current prod planning, disables all mutation except completion feedback. Probably a toggle within the existing app.
 6. **Simulation creation and teardown.** A "Simulate a new job" entry point; a new-tab flow with a full-copy fork of the preprod; TTL-based auto-cleanup.
 7. **Archive browsing.** A simple view of historical prod snapshots, read-only, useful for audit and undo recovery past the 5-minute window.
-

2. The Reasoning Path

This section captures the thinking journey. It is written for a future reader who might inherit this design and wonder *why not X*, *why not Y*. Preserving the discarded branches of exploration is often more useful than the final answer alone.

2.1 Starting point

The initial question was open: *"I want full production-like planning and settings (jobs, operators, machines...) and I want to create as many branches as I want to test different schedules, priorities, etc."* The user asked for a discussion with three specialist voices: UX, industrial planning, git-style versioning.

This framing suggested a rich multi-scenario system, similar to git branches applied to scheduling.

2.2 Initial specialist exploration

Each perspective contributed:

- **Industrial planning** confirmed that APS systems (SAP APO, Dassault Quintiq, Siemens Opcenter) have long had this concept, called *planning versions* or *scenarios*. It comes with known pitfalls: scenarios go stale quickly, users confuse environments, KPIs must be scenario-scoped.
- **UX** emphasized the mental model: users must never forget which environment they are in. Figma branching, Excel Scenario Manager, Google Docs versions were cited as concrete inspirations.
- **Software versioning** clarified which git primitives translate well (branch, commit, diff, tag) and which don't (textual 3-way merge, linear rebase) for a constrained-graph domain like scheduling.

At this stage, the default assumption was: *a scenario is a named, persistent, colored branch of the whole planning state.*

2.3 Six clarifying questions

Early answers narrowed the design:

1. Users: eventually multiple planners.
2. Scenario lifetime: hours, static (not continuously edited).
3. Promotion: yes, scenarios can become prod.
4. Count: typically 2-3 concurrent, no hard cap.
5. Baseline evolution during a scenario's life: irrelevant (scenarios short).
6. Scope: everything — jobs, priorities, operators, machines, groups, productivities.

These answers pointed to: **full-copy named snapshots** as the starter approach, with possible evolution toward overlays if usage grew.

2.4 Playground iterations

Three iterations of the HTML playground explored the UI:

- **V1** was confused: it included a side-by-side comparison mode (misreading "I'd want two tabs" as "I'd want a split view") and used a generic fake app layout that didn't resemble the real Flux Scheduler.
- **V2** corrected both: single window at a time, real Flux Toolbar and tab layout, banner-as-switcher pattern where the scenario name itself is the clickable dropdown (à la GitHub/Linear/Figma).
- **V3** explored an alternative: thin colored band at the top + floating action button (FAB) at the bottom-right. Production mode is visually neutral (no band, grey FAB); scenario mode adds

the band, a subtle viewport halo, and colored action buttons. The principle was: **ambient awareness through absence** — the normality is prod, a scenario is a marked exception.

2.5 The overlay detour

Multi-user considerations prompted a major architectural exploration: the **shared-reality + per-scenario overlay** model. Inspired by Kubernetes Kustomize, Docker layers, Figma component instances, and Airtable views.

The idea: entities are split into two layers.

- **Shared reality** — jobs, operators, machines, completions, events: data that flows in from the real world and should be visible to all scenarios instantly.
- **Scenario overlay** — priorities, task assignments, group configurations, productivities: the configuration the scenario is testing.

This would have solved the staleness problem elegantly (new jobs auto-propagate into every scenario) and allowed structural operations (task exclusions, additions, dependency rewrites) via Kustomize-style patches.

It was ultimately abandoned. When the real use cases became clear, none of them justified the complexity. The overlay model was a beautiful answer to a problem the user did not have.

2.6 The pullback to reality

The turning point: the user stepped back and described the actual workflow.

"An ADV answers the phone to a client asking 'can you fit a 15,000 brochure job before April 24?' The ADV creates a full copy of prod, adds the hypothetical job, plays with it to see if it fits. The plan is never promoted."

"The lead planner has ONE copy of the current production, call it a preprod. When a job becomes planifiable, it's added to this preprod. When the preprod is good, the planner pushes it to prod, overwriting everything except tile advancement."

These two narratives broke the generic scenario frame. They revealed:

- Two distinct patterns with different semantics, not variants of one concept.
- No multi-user collaboration on the same plan (one planner per company).
- No cross-scenario promotion conflicts (only one preprod gets promoted).
- No staleness worry (continuous sync handles it).

The rich scenario system was over-engineered for the actual needs.

2.7 Convergence to three environments

The final model emerged by naming each use case explicitly:

- The lead planner's ongoing work → **Preprod** (persistent, one per company).
- The ADV's feasibility tool → **Simulation** (ephemeral, N per company).
- The committed plan for the shop floor → **Prod** (frozen, read-only except completion).
- The safety net for promotion → **Archive** (historical snapshots, read-only).

No overlay, no branching metaphor, no switcher across colored scenarios. Three distinct environments with specific sync rules between them.

2.8 Validation as a planning-vs-execution pattern

The final insight came from the user: *"What we have today IS actually the preprod. The prod is read-only — you can only give completion feedback on it."*

This matched the classical industrial separation between **planning** (mutable, elaborated, debated) and **execution** (frozen, committed, followed). SAP calls it *Planned Order* vs *Process Order*; other APS systems call it *Preliminary Plan* vs *Committed Schedule*. Software calls it *dev* vs *prod*. In all cases, the ritual of *commit/publish/promote/release* sanctifies the boundary between "free to reshape" and "running for real".

This framing did more than validate the design — it revealed that the current app is already two-thirds of the way there. The preprod behavior already exists; what's missing is the explicit prod boundary and the promotion ritual.

3. Insights Worth Preserving

These are reusable lessons from the journey, larger than this specific feature:

- **Don't create generic abstractions; create specific ones.** A single concept ("scenario") covering two different patterns (*simulation* and *preprod*) invites confusion. Naming them distinctly and giving them different UIs, rules, and lifetimes is clearer and leads to simpler code.
- **Design for concrete use cases before designing for the general case.** The overlay architecture was technically elegant but solved problems the user did not have. Concrete use cases are the only honest source of requirements.
- **Planning/execution separation is an old, proven pattern.** Don't reinvent it; name it and borrow the known disciplines (commit ritual, one-way flows, reconciliation only for execution feedback).
- **The user's mental model is not necessarily the data model.** When the user says "I modify the job" and means "I change its machine assignment", the data mutation is localized to a

`TaskAssignment` row, not to the Job entity. The noun ("job") and the datum of the same name are not always the same thing.

- **Ambient awareness matters when context confusion has consequences.** In multi-environment systems, the cost of acting in the wrong environment determines how loud the context indicator must be. Free to be subtle when consequences are low; mandatory to be loud when they are high.
 - **Prefer full copies to overlays when use cases don't require synchronization.** Overlays are powerful but carry complexity (projection logic, structural operations, conflict semantics). Full copies are easier to reason about and sufficient when environments are few, single-user, and short-lived or clearly staged.
-

4. Inspirations Cited

Drawn from the discussion, grouped by category:

Industrial APS: SAP APO (planning versions), Dassault Quintiq, Siemens Opcenter — all have scenario/version concepts; all enforce planning-vs-execution separation.

Software versioning: Git (branches, commits, diff, tag — relevant; 3-way merge, linear rebase — not relevant to scheduling graphs).

Overlay systems: Kubernetes Kustomize (base + overlays + patches), Docker layers (add/remove/modify across layers), Figma component instances (master + per-instance overrides), Airtable views (table + view-specific filters/sorts), Notion database views.

UX patterns for context switching: Figma branching (non-developers using branch/review/merge), Linear cycles, Excel Scenario Manager, Google Docs named versions, GitHub repo/org picker (the context-name-is-the-switcher pattern), Slack workspace switcher.

Environmental discipline (ambient awareness): macOS screen-recording red bar, Jira sandbox banners, Chrome incognito coloring.

5. What Was Explored and Rejected

For completeness, decisions *not* taken:

Rejected choice	Reason
Rich multi-scenario system with names and colors	Actual use cases need two fixed environments + ephemeral bubbles, not a branching forest
Overlay/delta data model (shared reality + per-scenario overlay)	Elegant but unnecessary; no staleness pressure and no multi-user collaboration on same environment
Approver role separate from planner	The lead planner is the approver by role definition
Cross-editing of scenarios by multiple planners	One planner per company; collaboration happens through the promotion ritual, not simultaneous editing
Staleness / merge-conflict UX	With continuous sync and a single preprod, the problem does not arise
Side-by-side comparison UI between scenarios	Explicitly not needed — comparison is done in separate browser tabs if at all
Scenario KPI dashboards	Not needed given no comparison flow
Hot-patching prod directly during emergencies	Frequent promotion cadence makes the rebuild-in-preprod path fast enough (Option A chosen over Option B)
Retention / purge rules for archives	Archives are kept indefinitely; manual curation only if storage becomes a concern

6. Final State of the Mental Model

After the full journey:

- The current app = the future **preprod**. No behavioral change for the lead planner.
- A new **prod view** is added: read-only rendering of the committed plan, with completion feedback controls.
- A **promotion action** is introduced: the only moment the plan crosses from preprod to prod.
- A new **simulation entry point** is added for ADVs: creates a new browser tab with an isolated fork of the preprod.
- Two background workers glue the system together: reservoir→preprod job pushing, prod→preprod completion syncing.

The feature is much smaller than the original question suggested. That is the point.